# Unified Modelling Language (UML)


**Prof. Pradip K. Das**
Professor – Computer Sc. & Engineering Dept.
Head – Centre for Drone Technology
IIT Guwahati, Assam
e-mail: pkdas@iitg.ac.in

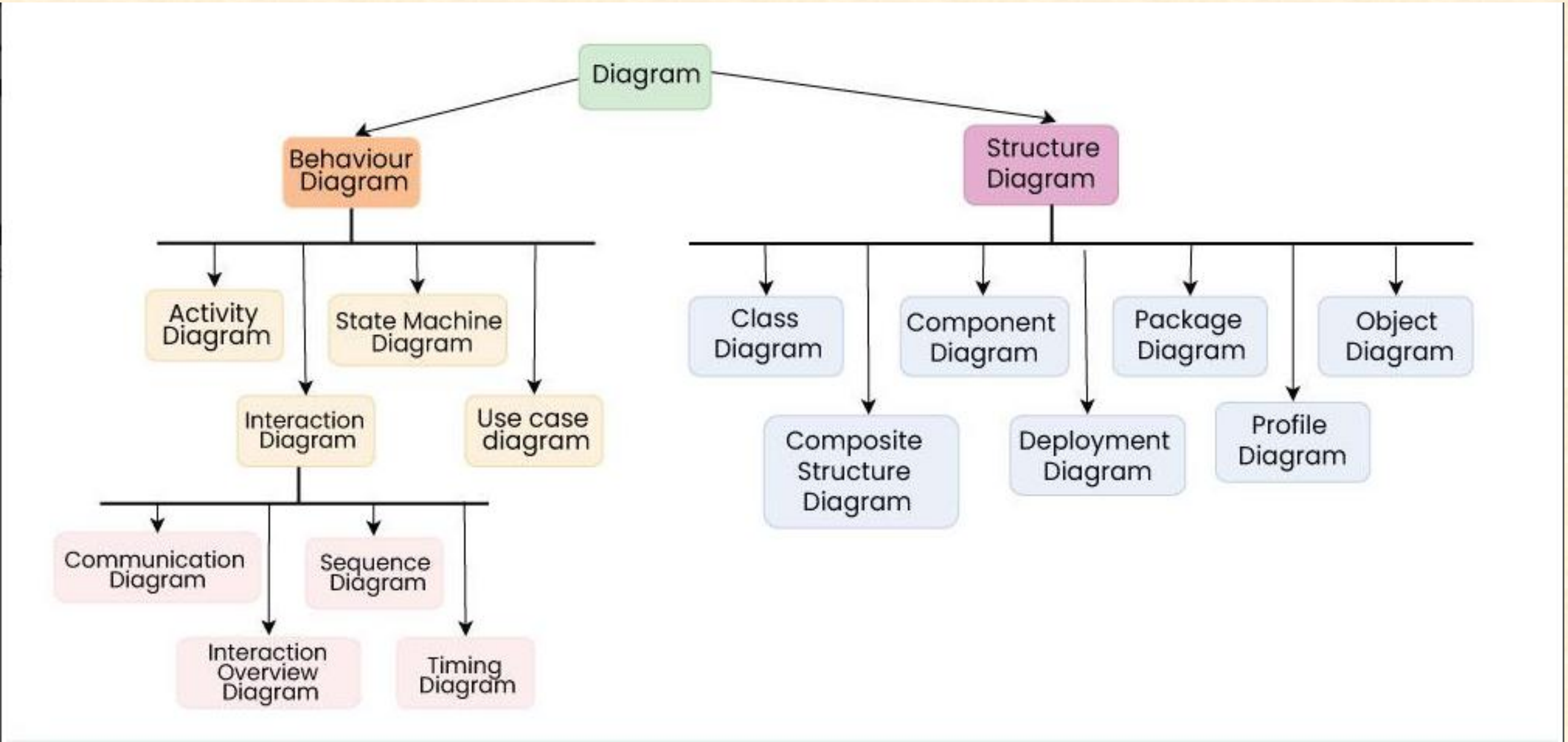
# What is UML?

- **Unified Modeling Language (UML)** is a standardized visual modeling language that is a versatile, flexible and user-friendly method for visualizing a system's design.
- Software system artifacts can be specified, visualized, built and documented with the use of UML.
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen and system architects with modelling, design and analysis.

# *Why do we need UML?*

- **UML** helps **visually represent** complex system designs for better collaboration.
- Enables **clear communication** among multiple teams in complex projects.
- Helps **non-programmers** understand system requirements and functionalities.
- Saves time by **visualizing processes, interactions and system structure**.

# Types of UML Diagrams



# Structural Diagrams

# Structural Diagrams

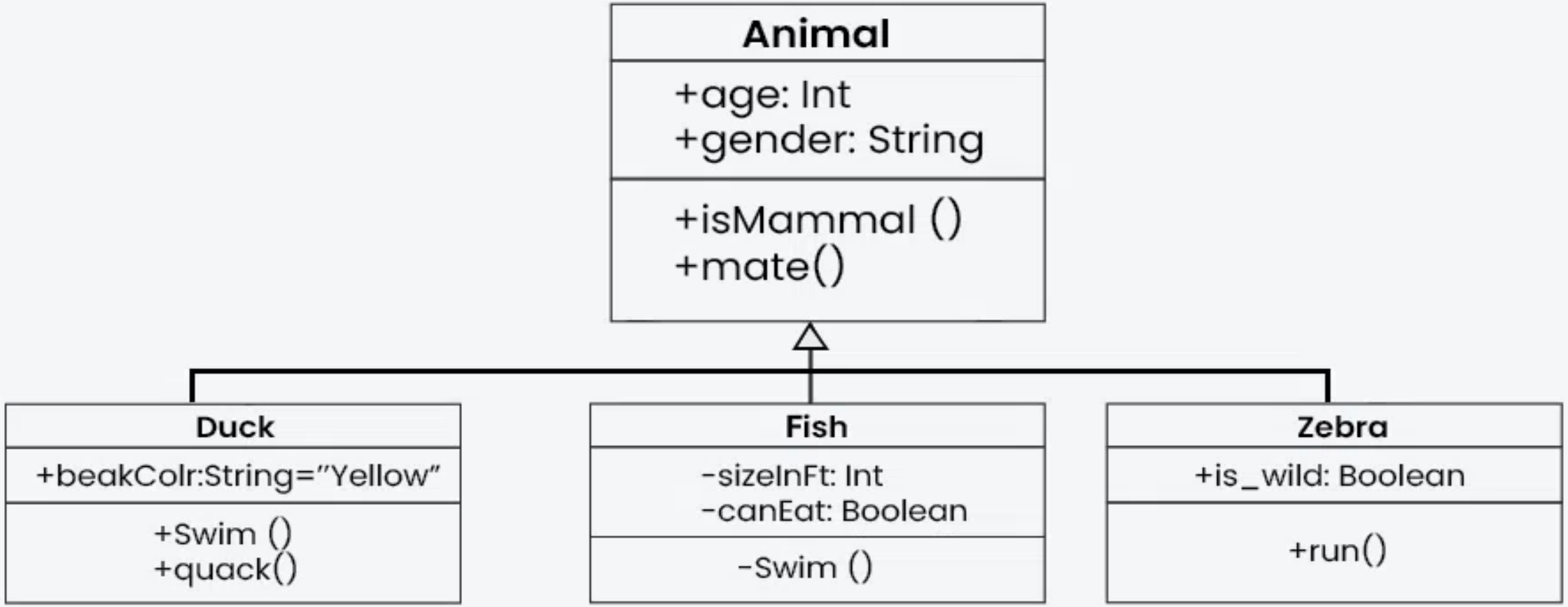
- Structural UML diagrams are visual representations that depict the static aspects of a system, including its classes, objects, components, and their relationships, providing a clear view of the system’s architecture.

- Structural UML diagrams include the following types:

# Class Diagram

- **Class Diagram** is the most widely used UML diagram.
- It serves as the **building block** of all object-oriented software systems.
- It depicts the **static structure** of a system by showing its classes, methods and attributes.
- It helps in identifying **relationships** between different classes or objects.

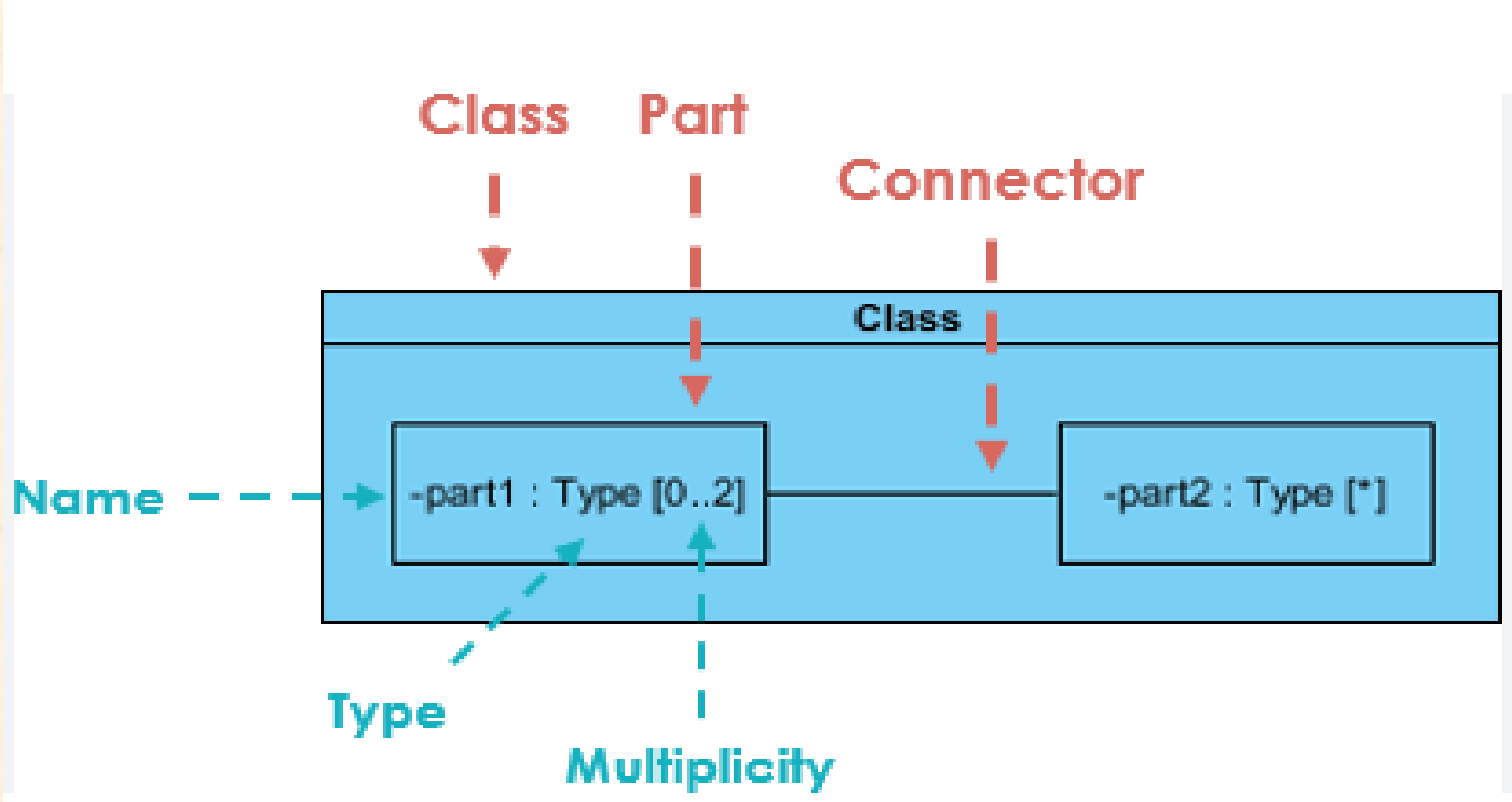
# Class Diagram



# Composite Structure Diagram

- **Composite Structure Diagrams** represent a class's internal structure and interaction points.
- They show relationships and configurations that define a classifier's behavior.
- They use **parts, ports** and **connectors** to depict internal structure.
- They can also model **collaborations** within a system.
- Unlike class diagrams, they focus on **individual parts** rather than the entire class.

# Composite Structure Diagram

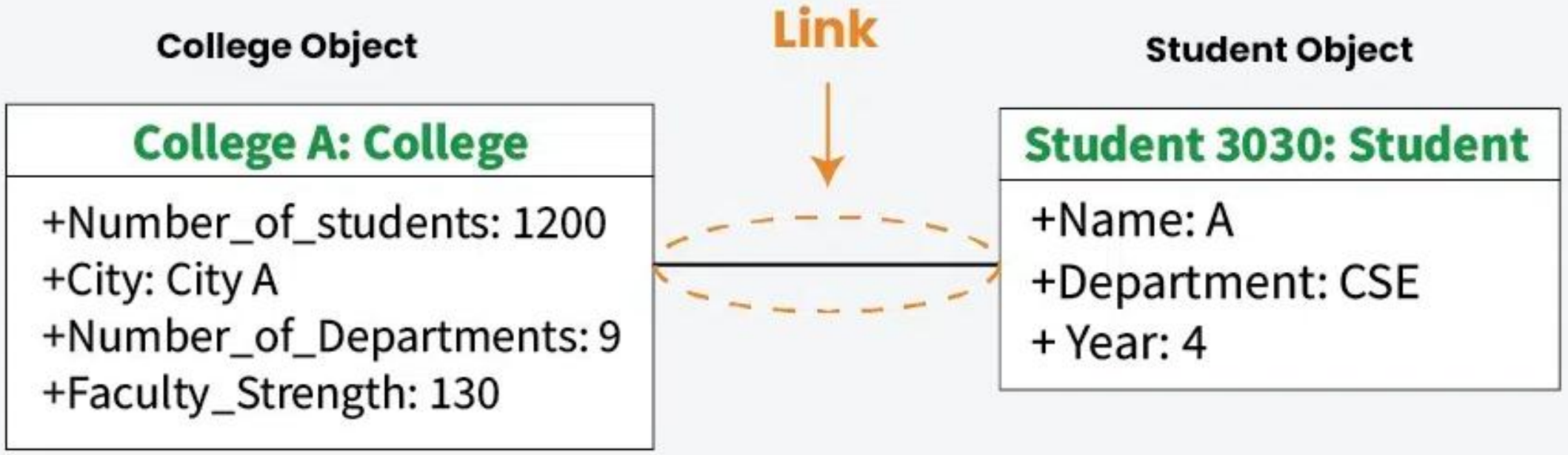


# Object Diagrams

- **Object Diagram** is like a **screenshot** of instances and their relationships in a system.
- It shows system **behavior *at a specific instant*** when objects are instantiated.
- Similar to a class diagram but focuses on **instances** instead of classes.
- **Class diagrams** depict classifiers and relationships, while **object diagrams** show specific instances at a point in time.

# Object Diagrams

An object diagram using a link and 2 objects

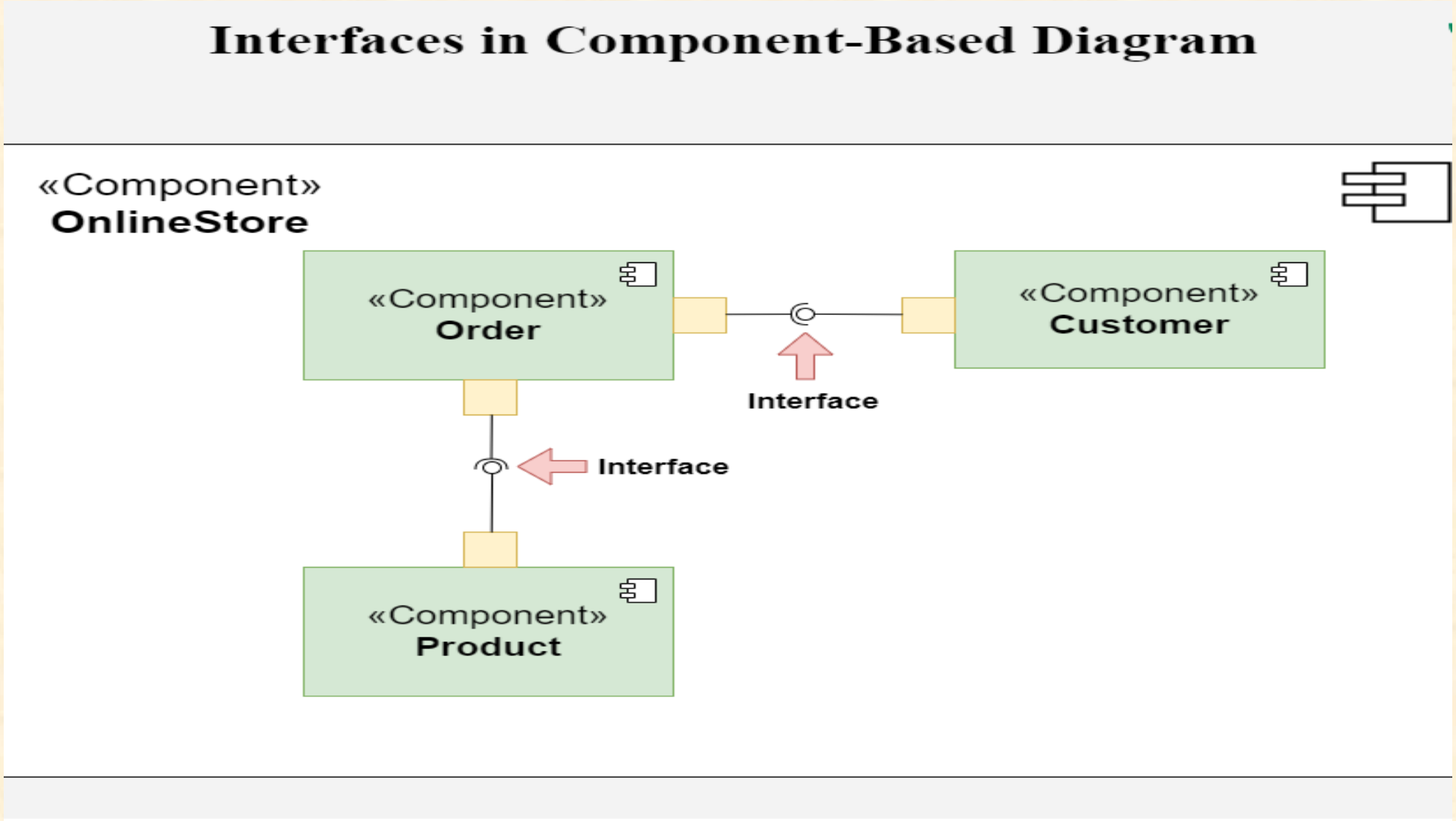



**An object of class Student is linked to an object of class College.**

# Component Based Diagram

- **Component Diagrams** show the **organization** of physical components in a system.
- They help model **implementation details** and structural relationships.
- They ensure **functional requirements** are covered in development.
- Essential for **designing complex systems**.
- Components communicate using **interfaces**.

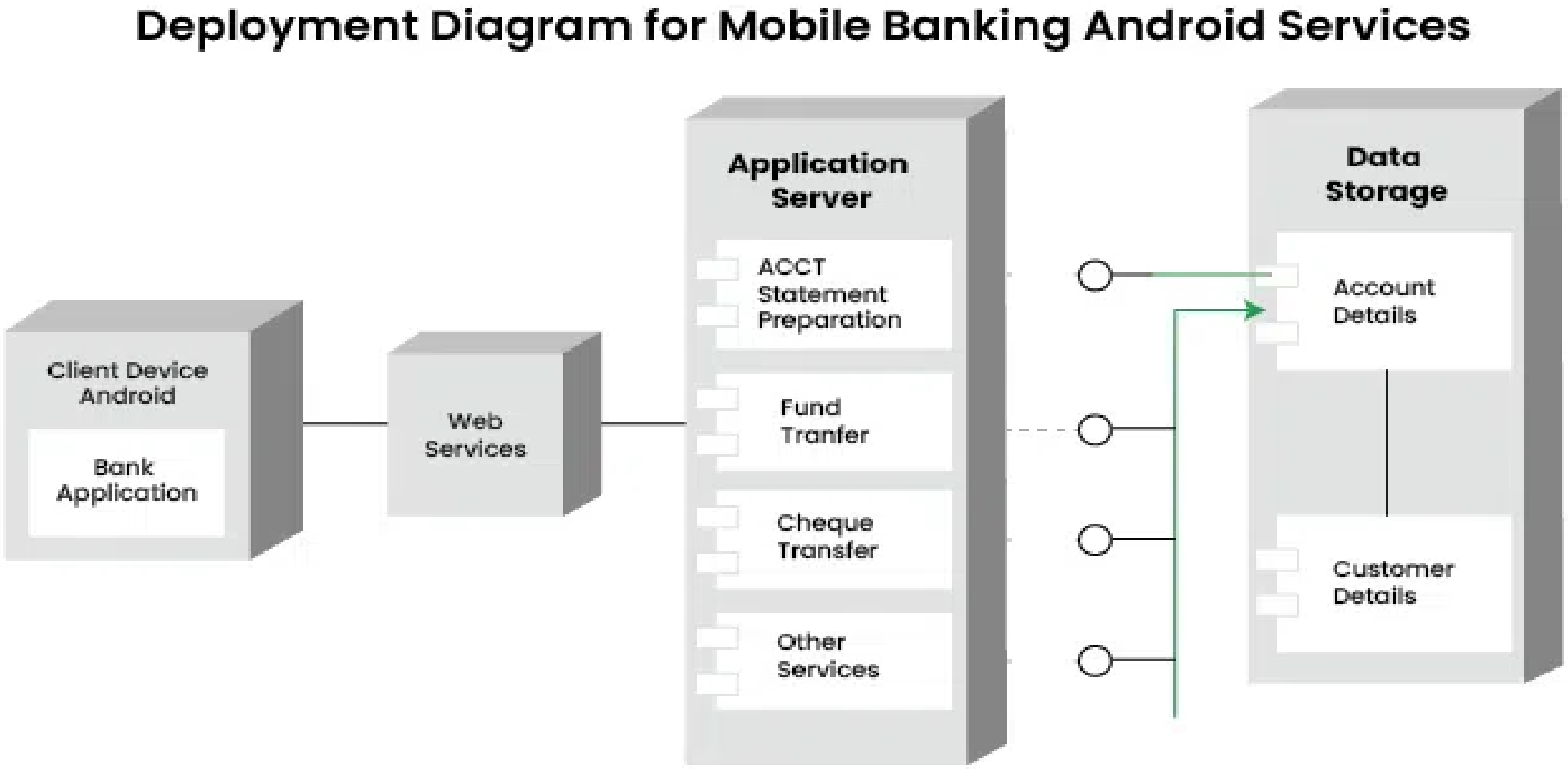
# Component Based Diagram



# Deployment Diagram

- **Deployment Diagrams** represent **system hardware and software**.
- Show **hardware components** and the **software running on them**.
- Illustrate **software distribution across targets**.
- **Artifacts** are information generated by system software.
- Used for **software deployment across multiple machines**.

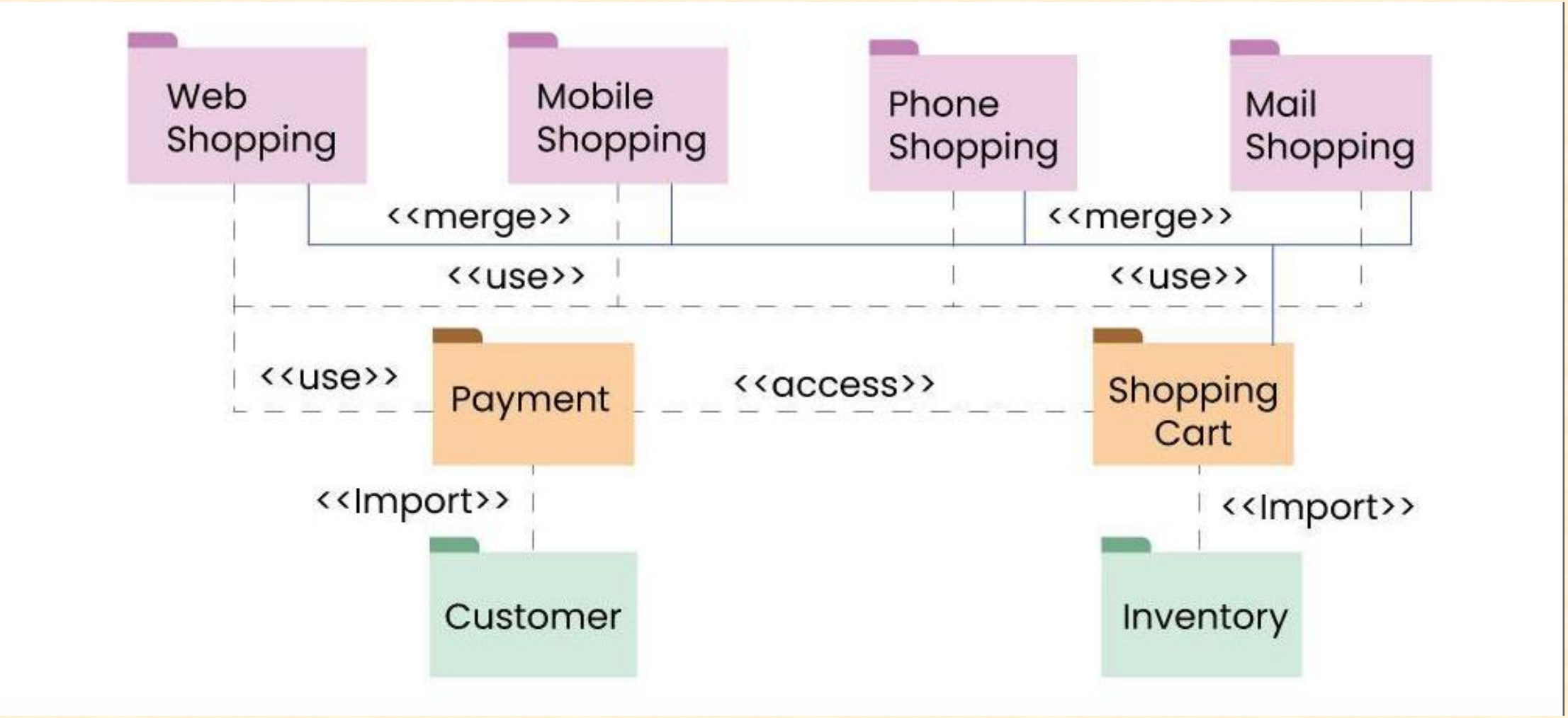
# Deployment Diagram



# Package Diagram

- **Package Diagrams** show **organization of packages and their elements**.
- Depict **dependencies** and **internal composition** of packages.
- Help **group UML diagrams** for better clarity.
- Mainly used to **organize class and use case diagrams**.

# Package Diagram



# Behavioral Diagrams

# Behavioral Diagrams

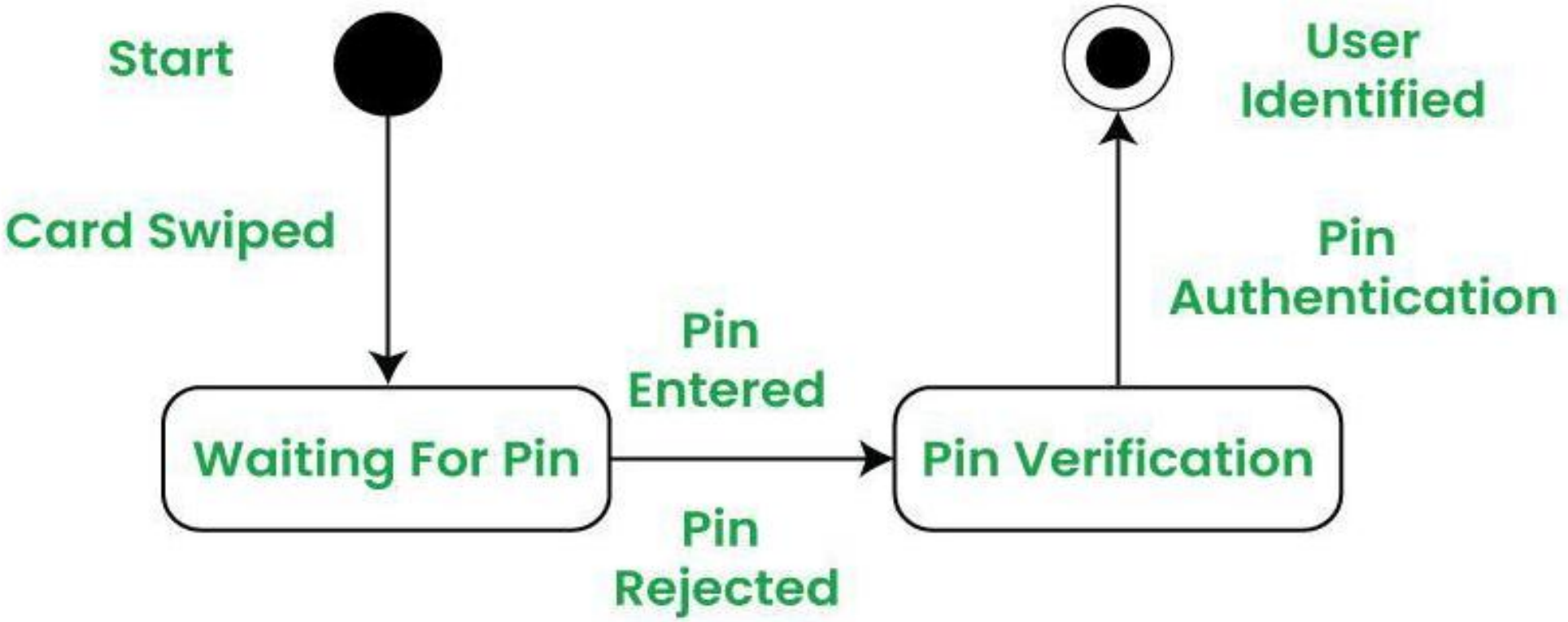
- Behavioral UML diagrams are visual representations that depict the dynamic aspects of a system, illustrating how objects interact and behave over time in response to events.

# State Machine Diagrams

- **State Diagram** represents a system's **condition at finite instances**.
- A **behavioral diagram** showing **finite state transitions**.
- Also called **State Machine** or **State-Chart Diagram**.
- Models a class's **dynamic behavior** in response to time and external stimuli.

# State Machine Diagrams

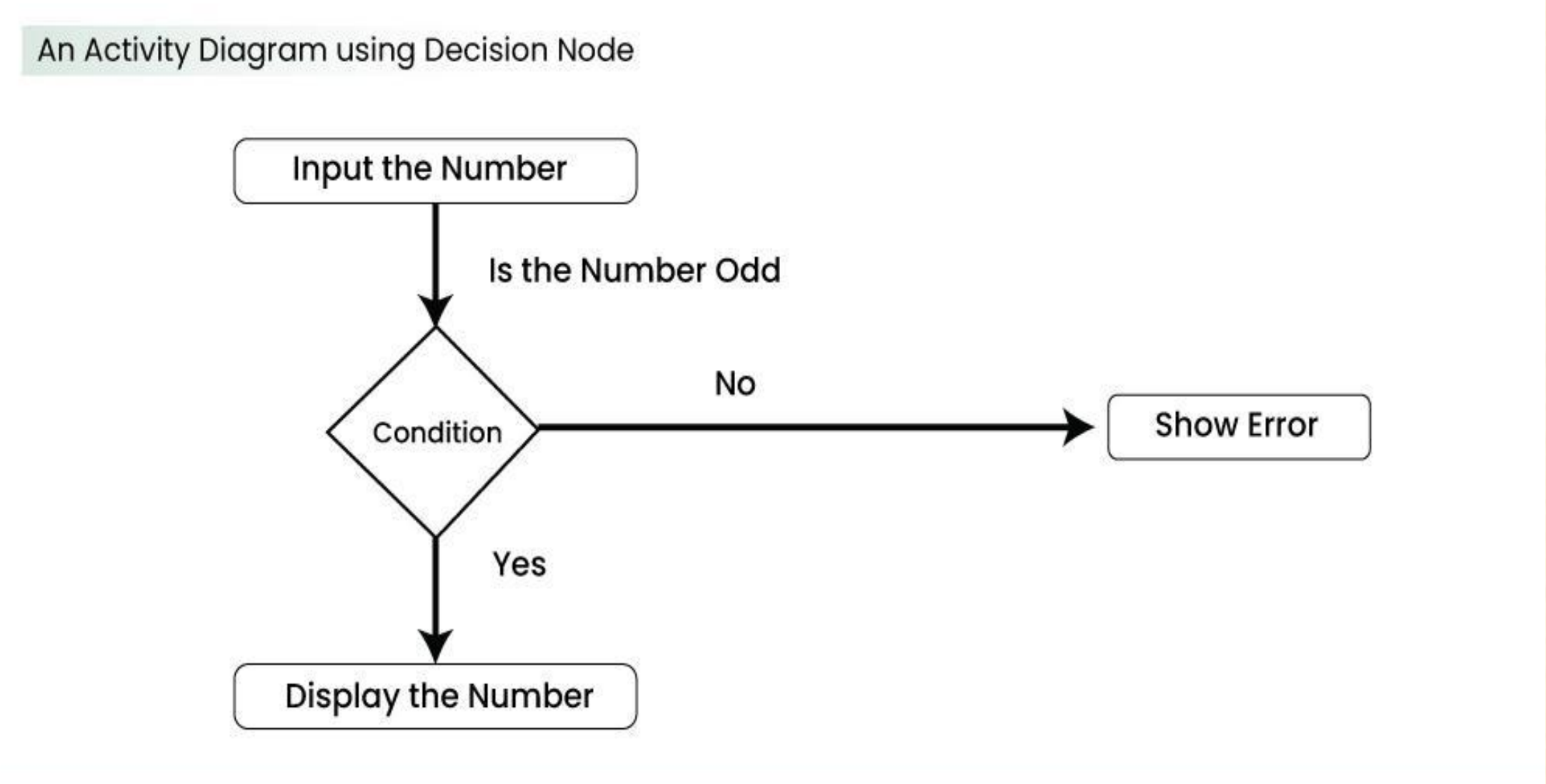
A State Machine Diagram for user verification



# Activity Diagrams

- **Activity Diagrams** show **flow of control** in a system.
- Represent **steps in a use case execution**.
- Model **sequential and concurrent activities**.
- Focus on **flow conditions and sequence**.
- Depict **event triggers** in a system.

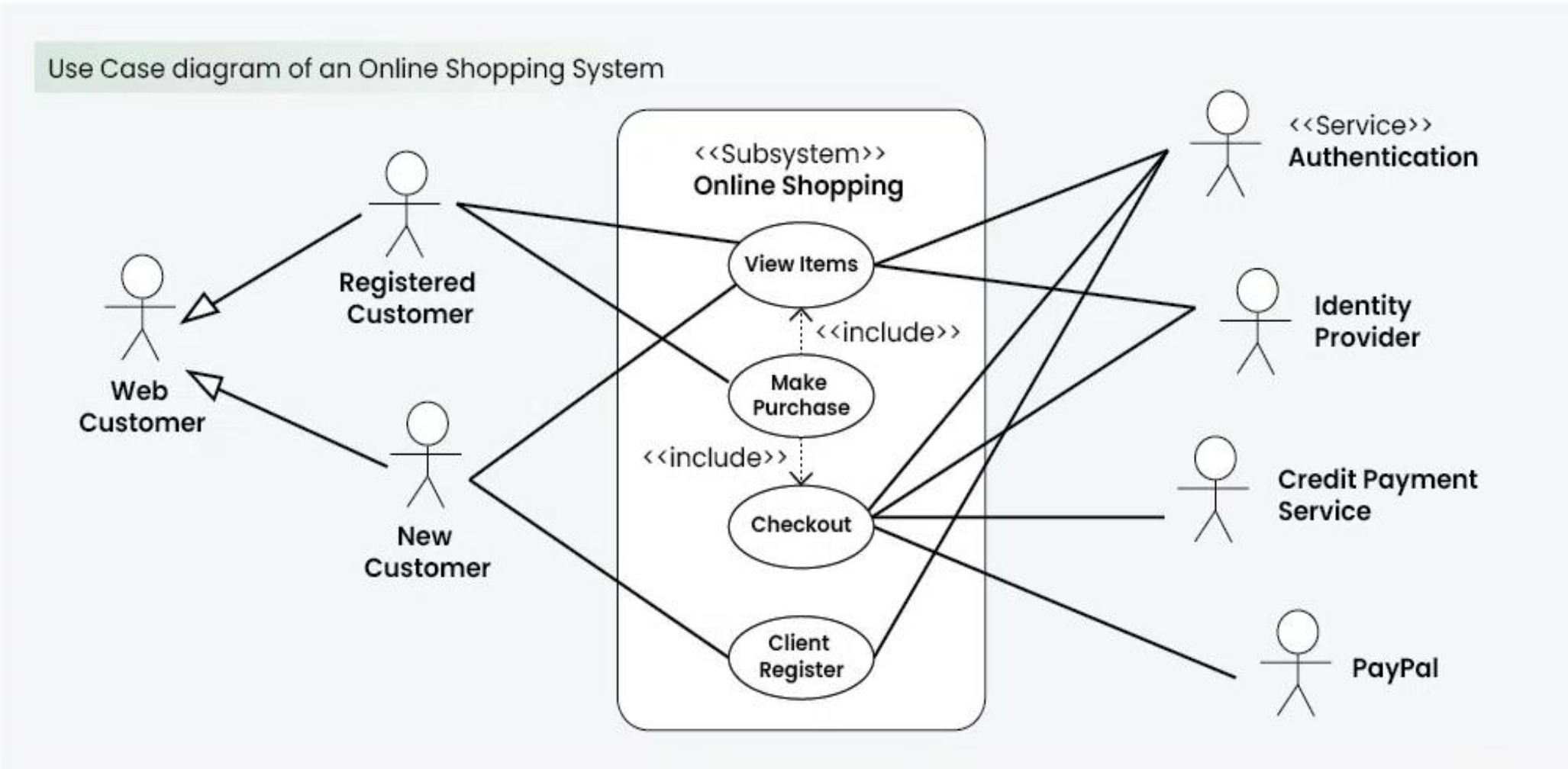
# Activity Diagrams



# Use Case Diagram

- **Use Case Diagrams** depict system **functionality** and interactions with **actors**.
- Illustrate **functional requirements** of the system.
- Represent **scenarios** where the system is used.
- Provide a **high-level view** without implementation details.

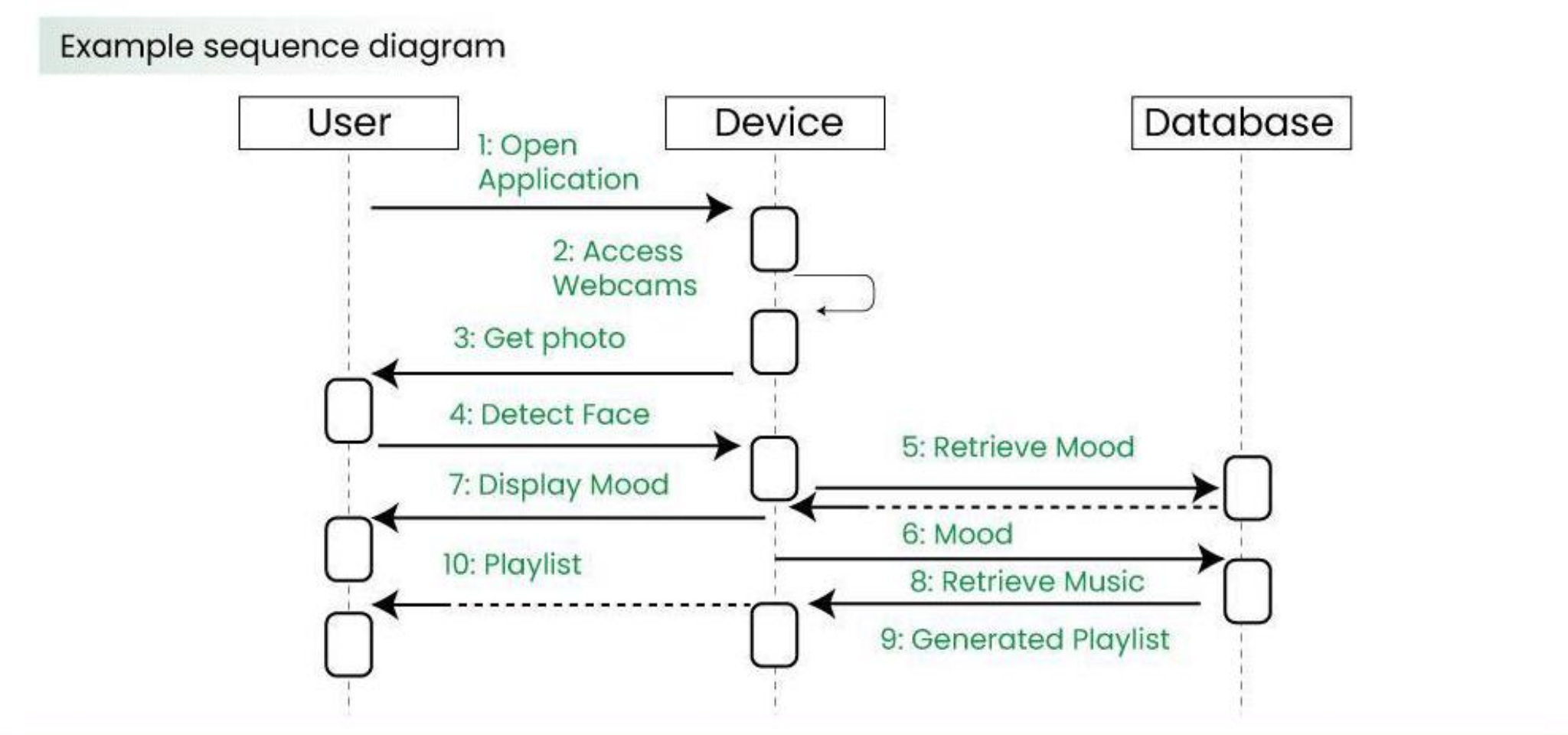
# Use Case Diagram



# Sequence Diagrams

- **Sequence Diagram** shows object interactions in **sequential order**.
- Also called **event diagrams** or **event scenarios**.
- Describes **how and in what order** objects function.
- Used by **businesses and developers** to document and understand system requirements.

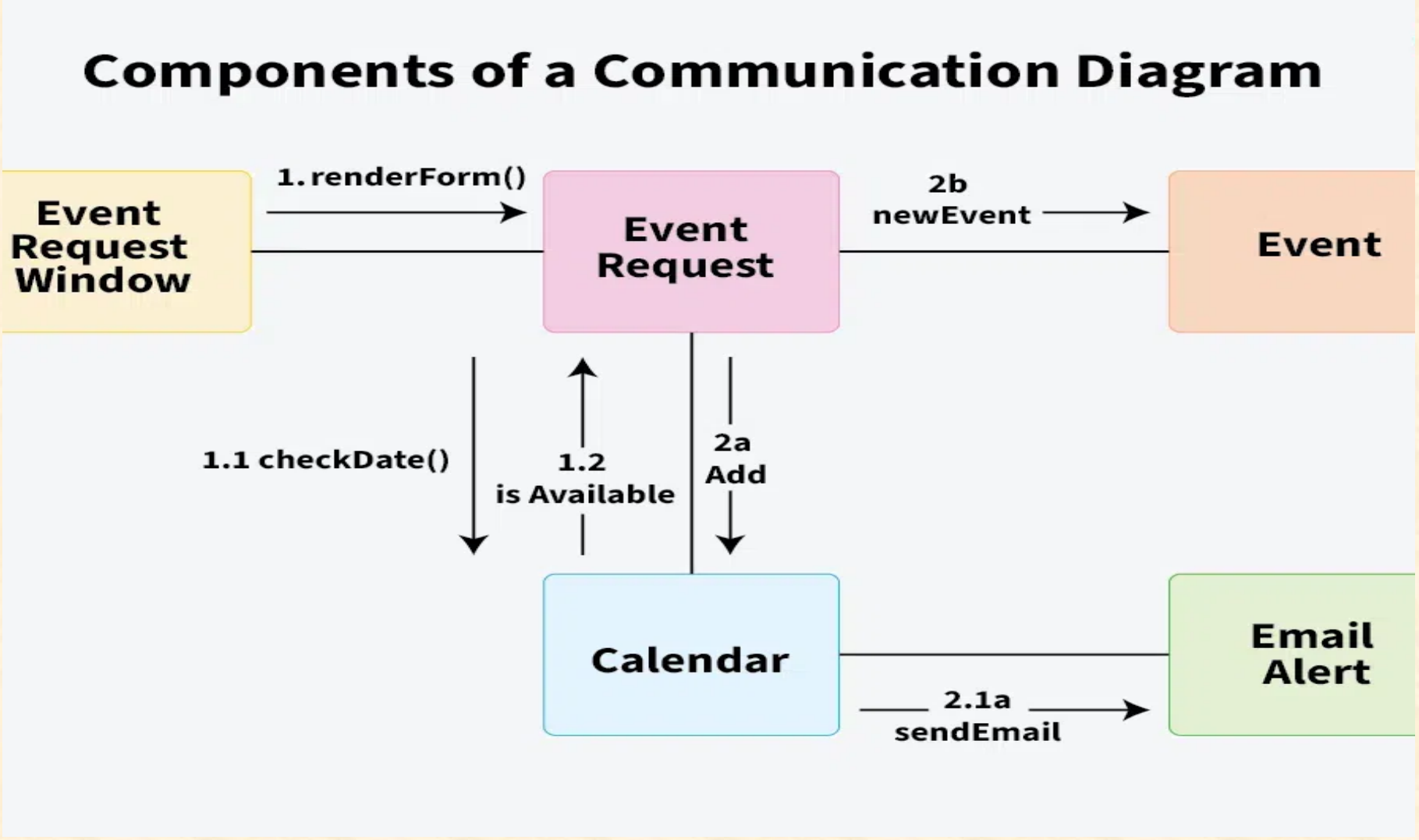
# Sequence Diagrams



# Communication Diagram

- A Communication Diagram (known as Collaboration Diagram in UML 1.x) is used to show sequenced messages exchanged between objects.
- A communication diagram focuses primarily on objects and their relationships.
- We can represent similar information using Sequence diagrams, however communication diagrams represent objects and links in a free form.

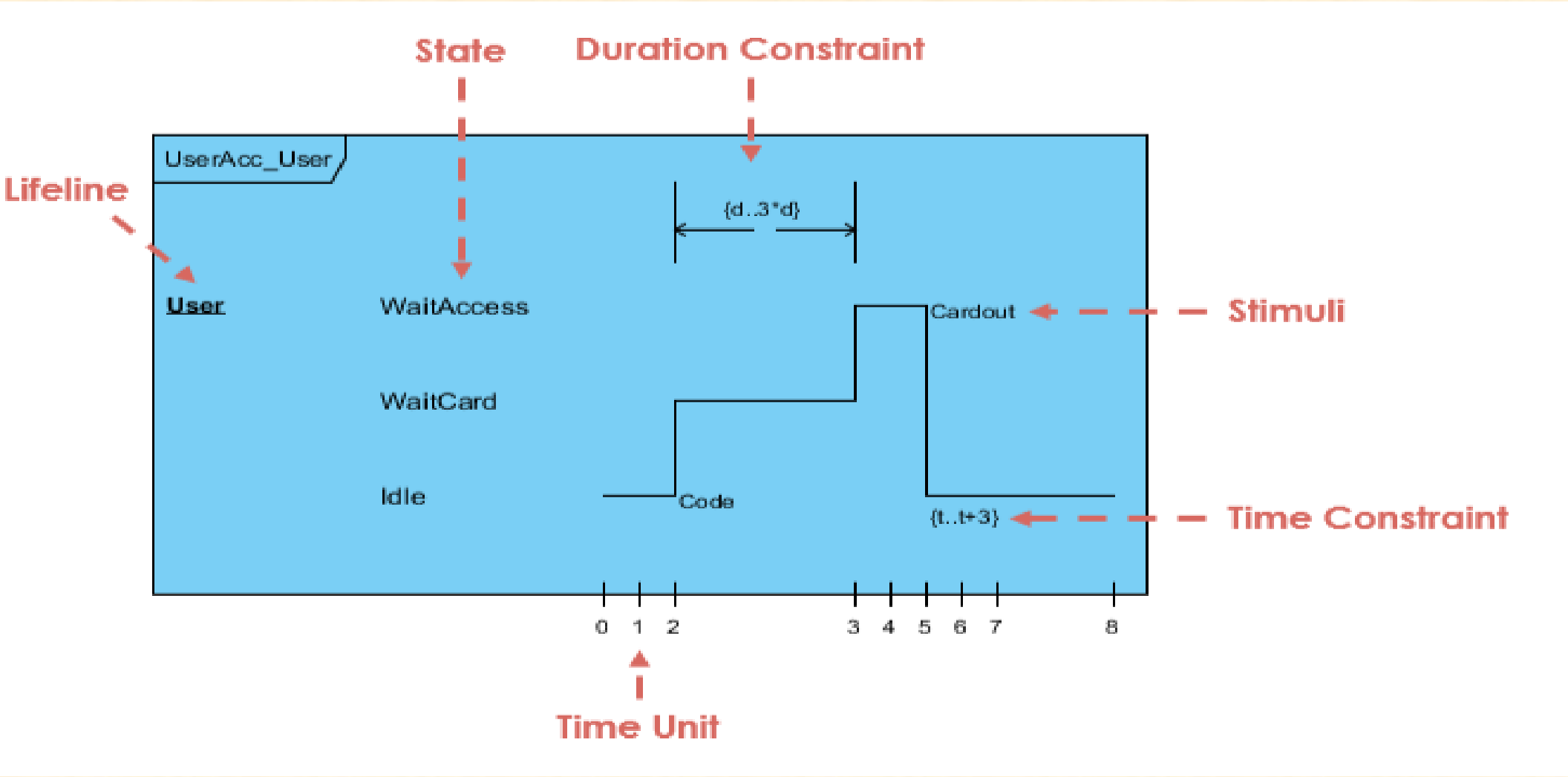
# Communication Diagram



# Timing Diagram

- **Timing Diagrams** are a special type of **Sequence Diagram**.
- Depict **object behavior over a time frame**.
- Show **time and duration constraints** affecting state changes.
- Represent **state and behavior transitions** over time.

# Timing Diagram

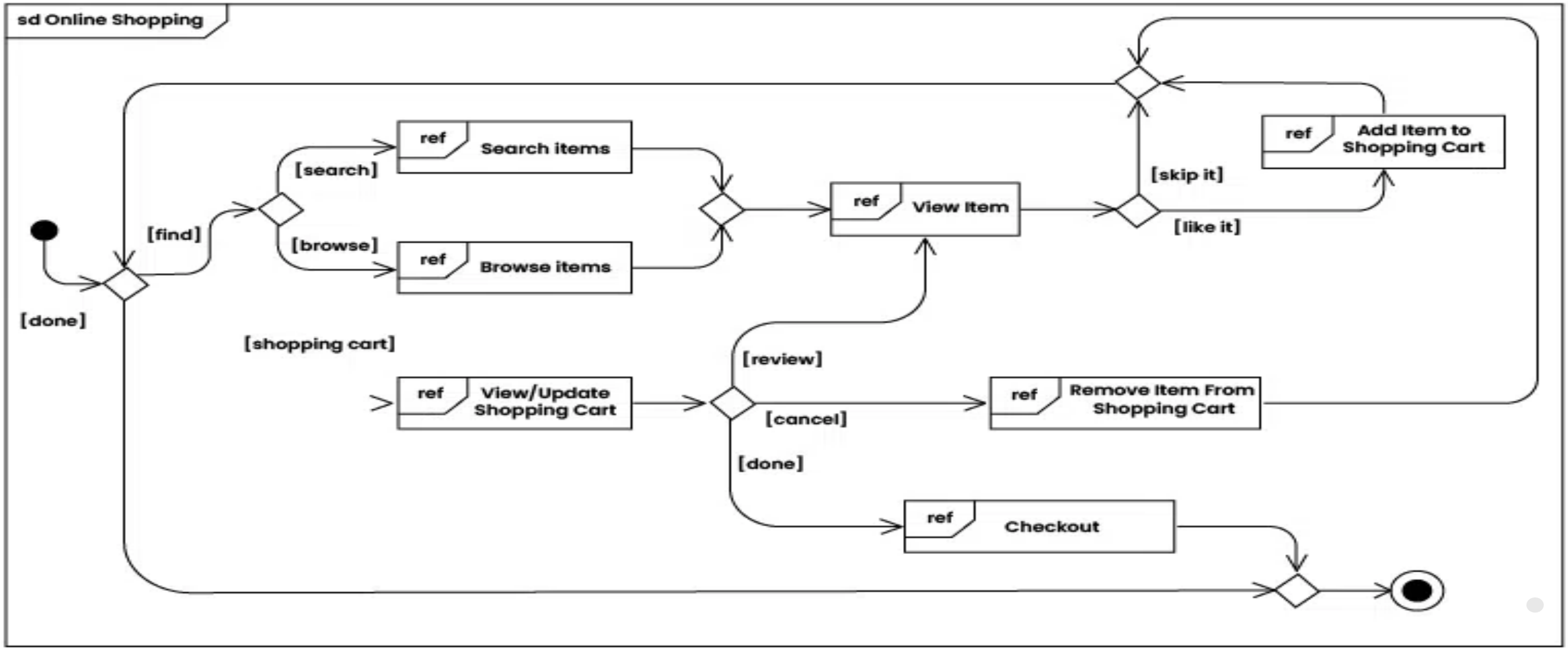


# Interaction Overview Diagrams

- **Interaction Overview Diagram (IOD)** is a type of UML diagram.
- It illustrates the **flow of interactions** between system elements.
- Provides a **high-level overview** of interactions.
- Shows the **sequence of actions, decisions and interactions** between components or objects.

# Interaction Overview Diagrams

## Example of Interaction overview Diagram



Yup! That’s all!